Design Patterns 2

Programare Orientată pe Obiecte

Clasificare

Se disting trei categorii:

1. Creational patterns: scheme “creaţionale“
   - generează obiectele necesare
   - mecanisme de creare a obiectelor
   - Ex. *Singleton*, *Factory* etc

2. Structural patterns: scheme structurale
   - definesc relaţii între entităţi
   - grupează mai multe obiecte în structuri mai mari.
   - Decorator, Adapter, Composite, Proxy etc.

3. Behavioral patterns: scheme de interacţiune
   - definesc comunicarea între entităţi
   - *Iterator*, *Visitor*, *Observer*, *Command*, Mediator etc.

Design Pattern

Observer

Programare Orientată pe Obiecte

Observer Pattern

- Behavioral pattern (comportamental)
  - o organizare mai bună a comunicaţiei dintre clase, funcţie de rolurile/comportamentul lor.

- Subiect - Obiectul observat :
  - un obiect care poate suferi diverse modificări
  - conţine o listă de referinţe la obiecte *observator*
  - la producerea unui eveniment, apelează o anumită metodă a obiectelor *observator* înregistrate la el.

- Observatori - unul sau mai multe obiecte observator:
  - trebuie anuntate imediat de orice modificare în obiectul *observat*, pentru a realiza anumite acţiuni.

- Clasele Observer **observă** modificările/acţiunile clasei Subject

Observer Pattern

- Mecanism de decuplare a obiectelor observabile,
  - care generează evenimente
- de obiectele observator,
  - care recepţionează evenimentele respective

- Relaţia dintre o clasă JFC generator de evenimente şi o clasă ascultător (receptor) care reacţionează la evenimente este similară relaţiei dintre o clasă observată şi o clasă observator!

Exemplu:

- un buton *JButton* are rolul de obiect *observat*
- o clasă care implementează interfata *ActionListener* (contine metoda *actionPerformed*) are rolul de *observator*.

Exemplu API Java

- Schema observat-observator a generat în Java
  - clasa *Observable*
  - interfaţa *Observer*
  - din pachetul *java.util*
- Programatorul va defini una sau mai multe clase cu rol de observator, compatibile cu interfaţa *Observer*.
- Motivul existenţei acestei interfeţe: în clasa *Observable* (pentru obiecte observate) există metode cu argument de tip *Observer* pentru menţinerea unui vector de obiecte observator.

Exemplu:

```
public void addObserver(Observer o) {
    //adauga un nou observator la lista
    if (!observers.contains(o))
            // "observers" este vector din clasa Observable
        observers.addElement(o);
 }
```

Observer - Observable

Interfaţa *Observer:*

```
public interface Observer {
        void update(Observable o, Object arg);
            // o este obiectul observat
}
```

- apelată de un obiect observat la o schimbare în starea sa care poate interesa obiectele observator înregistrate anterior

Clasa *Observable*:

- nu este abstractă dar nici nu este direct utilizabilă
- se va defini o subclasă a sa, adăugând o serie de metode specifice aplicaţiei care apelează din superclasă metodele:
  - setChanged
  - notifyObservers
    - apelează metoda ***update*** pentru toţi observatorii introduşi în vectorul *observers*

Clasa Observable - fragment

```
public class Observable {
    private boolean changed = false; //daca s-a produs o schimbare
    private Vector obs;                  // lista de obiecte observator
    public void addObserver(Observer o) {
     // adauga un observator la lista
       if (!obs.contains(o))
          obs.addElement(o);
    }
    public void notifyObservers(Object arg) {
     // notificare observatori de schimbare
       if (!changed)   return;
       for (int i = obs.size()-1; i>=0; i--)
            ((Observer)obs.elementAt(i)).update(this, arg);
    }
    protected void setChanged() {
        // comanda modificare stare obiect observat
        changed = true;
    }
  . . .
```

Exemplu utilizare Observable-Observer

- Exemplu de definire a unei subclase pentru obiecte observate:

```
class MyObservable extends Observable {
    public void change() {    // metoda adaugata in subclasa
        setChanged();              // din clasa Observable
        notifyObservers();
            // anunta observatorii ca a aparut o schimbare
    }
}
```

- Clasa observator: metoda *update* adaugă câte un caracter # pe ecran (la fiecare apelare de către un obiect MyObservable).

```
class ProgressBar implements Observer {
    public void update( Observable obs, Object x ) {
            System.out.print('#');
    }
}
```

Exemplu - continuare

- Utilizare a obiectelor observat-observator definite anterior:

```
public static void main(String[ ] arg) {
    int n=1000000000, m=n/100;
    ProgressBar bar= new ProgressBar();
    MyObservable model = new MyObservable();

    model.addObserver(bar);

    for (int i=0;i<n;i++)
          // modifica periodic stare obiect observat
          if ( i%m==0)
            model.change();
  }
```

Exemple JDK

- Interfeţele *ActionListener, ItemListener* s.a.
  - rol similar cu interfaţa *Observer*
- metodele "actionPerformed" s.a.
  - corespund metodei "update".
- Programatorul de aplicaţie defineşte clase ascultător compatibile cu interfetele "xxxListener", clase care implementează metodele de tratare a evenimentelor
- Corespondenţa dintre metodele celor două grupe de clase:
- addObserver
  - addActionListener
  - addChangeListener
- notifyObservers
  - fireActionEvent
  - fireChangeEvent
- update
  - actionPerformed
  - stateChanged

Design Pattern

Command

Programare Orientată pe Obiecte

Design pattern-ul *Command*

- *Behavioral pattern*: legat de interacţiunea dintre componente, de felul în care se efectuează apelurile.
- Este utilizat un obiect pentru a reprezenta şi încapsula toate informaţiile necesare pentru a apela mai târziu o anumită metodă:
  - Numele metodei, obiectul de care ţine metoda şi valorile parametrilor metodei.
- Permite:
  - actiuni (comenzi) multiple
  - decuplarea alegerii operatiei executate de locul unde este emisă comanda.

- foloseste o interfată generală, cu o singură functie, de felul următor:

```
public interface Command {
    public void execute(); // execută o actiune nedefinită încă
}
```

Command pattern – descriere

- Entităţi:
  - Command
  - Receiver
  - Invoker
  - Client
- Un obiect *command* are un obiect *receiver* şi invocă o metodă a *receiver*-ului într-un mod specific clasei acelui *receiver*.
- *Receiver*-ul realizează acţiunea.
- Un obiect *command* este transmis separat unui obiect *invoker*, care invocă comanda, şi opţional, realizează arhivarea execuţiei comenzii.
- Obiectul *invoker* poate primi orice obiect *command*.
- Atât un obiect *invoker* cât şi obiectele *command* sunt păstrate de un obiect *client*.
- *Clientul* conţine decizia legată de ce comandă să se execute şi la ce moment. Pentru a executa o comandă, el transmite obiectul *command* obiectului *invoker*.

Tipuri de componente (roluri):

- **Invoker** – cere execuţia unor comenzi
  - apelează acţiuni - metode oferite de obiectele de tip *Command*
  - poate menţine, o *listă a tutoror comenzilor aplicate* pe obiectul (obiectele) comandate.
  - primeşte clase *Command* pe care să le invoce
- **Receiver**
  - clasa asupra căreia se face apelul
  - conţine implementarea efectivă a ceea ce se doreşte executat
- **Command** - obiectele pentru reprezentarea comenzilor implementează această interfață sau o extind (dacă este clasă abstractă)
  - conţin metode cu nume sugestiv pentru executarea acţiunii comenzii - execute()
  - conţin referinţe către *receivers* pentru a realiza comanda
  - *concrete command* - implementări/subclase
- **Client** – crează comenzile concrete şi setează receiver-ul lor

Exemplu

- Un “switch” cu două comenzi: aprinderea/stingerea luminii.
- Acest switch poate fi folosit cu orice alt gen de acţiune pentru un alt aparat, nu doar pentru aprinderea-stingerea luminii, metoda Switch-ului poate primi orice subclasă Command ca parametru.
- Exemplu: îl putem configura pentru pornirea unui motor.

```
import java.util.List;
import java.util.ArrayList;

 /* interfata Command */
public interface Command {
   void execute();
}
```

Exemplu

```
/* Invoker */
public class Switch {
   private List<Command> history = new ArrayList<Command>();
    public void storeAndExecute(Command cmd) {
            history.add(cmd);           // optional
            cmd.execute();
   }
}

/* Receiver */
public class Light {
   public void turnOn() {
      System.out.println("The light is on");
   }
   public void turnOff() {
      System.out.println("The light is off");
   }
}
```

Exemplu

```
 /* Comanda pentru aprinderea luminii – comanda concreta 1 */
public class FlipUpCommand implements Command {
    private Light theLight;
    public FlipUpCommand(Light light) {
      theLight = light;
   }
    public void execute(){
      theLight.turnOn();
   }
}
 /* Comanda pentru stingerea luminii – comanda concreta 2 */
public class FlipDownCommand implements Command {
   private Light theLight;
   public FlipDownCommand(Light light) {
      theLight = light;
   }
    public void execute() {
      theLight.turnOff();
   }
}
```

Exemplu

```
/* Clasa test sau client */
public class PressSwitch {
   public static void main(String[] args){
      Light lamp = new Light();
      Command switchUp = new FlipUpCommand(lamp);
      Command switchDown = new FlipDownCommand(lamp);
      Switch mySwitch = new Switch();
       try {
         if ("ON".equalsIgnoreCase(args[0]))
               mySwitch.storeAndExecute( switchUp );
         else if ("OFF".equalsIgnoreCase(args[0]))
                mySwitch.storeAndExecute (switchDown );
             else
                  System.out.println("\"ON\" sau \"OFF\" !!!");
      } catch (Exception e) {
               System.out.println("Introduceti argumentele!!");
      }
   }
}
```

Design pattern-ul *Command*

- In Swing există mai multe interfeţe "ascultător" corespunzătoare interfeţei “Command”. Exemplu:

```
public interface ActionListener {
    public void actionPerformed( ActionEvent ev);
}
```

- Un obiect dintr-o clasă ascultător este un obiect "comandă" ce realizează o acţiune.
  - singura legătură dintre partea de interfaţă grafică a aplicaţiei şi partea de logică specifică aplicaţiei (tratarea evenimentelor).
  - Se realizează astfel decuplarea celor două părţi, ceea ce permite modificarea lor separată şi chiar selectarea unei anumite acţiuni în cursul execuţiei.

Exemplu fără command pattern

```
 …
JMenuBar mbar = new JMenuBar();
setJMenuBar (mbar);
JMenu mFile = new JMenu ("File");
mbar.add (mFile);
JMenuItem open = new JMenuItem ("Open");
    // optiune meniu vertical
JMenuItem exit = new JMenuItem ("Exit");
    // optiune meniu vertical
mFile.add (open);
mFile.addSeparator( );
mFile.add (exit);
open.addActionListener (this);
exit.addActionListener (this);
…..
```

```
// Tratarea evenimentelor
public void actionPerformed (ActionEvent e) {
  Object source = e.getSource(); // sursa evenimentului
  if ( source == open)  fileOpen( );// actiune asociata lui "Open"
  else if (source == exit) System.exit(0); // actiune asociata lui "Exit"
}
```

Exemplul cu *command pattern*

- *o interfaţă "Command”*

```
public interface Command {
    public void execute();
}
```

- *definirea mai multor clase care implementează această interfaţă:*

```
class FileExitCmd  extends JMenuItem implements Command {
    public FileExitCmd (String optname) {
            super(optname);  // nume optiune (afisat in meniu)
    }
    public void execute () {
            System.exit(0);          // actiune asociata optiunii exit
    }
}

class FileOpenCmd  extends JMenuItem implements Command
{
    public FileOpenCmd (String optname) {
            super(optname);  // nume optiune (afisat in meniu)
    }
```

Exemplul cu *command pattern*

```
    public void execute () {
            fileOpen();      // actiune asociata optiunii open
    }
}

class CmdListener implements ActionListener {
      public void actionPerformed (ActionEvent e) {
            Command c = (Command) e.getSource();
             c.execute();
      }
} // Invoker
    . . .
    ActionListener cmd = new CmdListener();
    open.addActionListener (cmd);
    exit.addActionListener (cmd);
```

- Metoda "execute" : metodă polimorfică
  - selectarea unei implementări sau alta se face în funcţie de tipul variabilei ***c***, deci de tipul obiectului care este sursa evenimentului!

Exemplu utilizare Command pattern

- în programarea meniurilor si barelor de instrumente ("toolbar"), prin utilizarea obiectelor "actiune” - Action
- implementează interfata *Action*.
- O bară de instrumente contine mai multe butoane cu imagini (obiecte *JButton*), dar are acelasi rol cu o bară meniu: selectarea de actiuni de către operator.
- In loc să se adauge celor două bare obiecte diferite (dar care produc aceeasi actiune) se adaugă un singur obiect, de tip *Action*, care contine o metodă *actionPerformed*; câte un obiect pentru fiecare actiune selectată.
- Interfata *Action* ⇔ interfaţa *Command*
- metoda *actionPerformed* ⇔ metoda *execute*.
- Interfata *Action* extinde interfata *ActionPerformed* cu două metode *setValue* şi *getValue*:
  - stabilirea si obtinerea proprietătilor unui obiect actiune (text afisat în optiune meniu, imagine afisată pe buton din bara de instrumente, o scurtă descriere ("tooltip") a "instrumentului" afisat)

Utilizare

- Facilitează construirea de componente generale care au nevoie să delege, sau să execute apeluri de metode fără să fie necesar să cunoască clasa metodei sau parametrii metodei în momentul în care realizează aceasta!

- Utilizarea unui obiect invoker permite
  - păstrarea evidenţei execuţiei comenzilor
  - implementarea mai multor tipuri de comenzi care sunt gestionate de invoker
- fără să fie necesar ca clientul să ştie că există această arhivare sau aceste tipuri!

Design Pattern
Visitor

Programare Orientată pe Obiecte

Visitor pattern

- *Behavioral pattern* - comportamental

- Modalitate de a *separa un algoritm* de *structura* pe care acesta operează.

Putem adăuga noi:

- posibilităţi de prelucrare a structurii, fără să o modificăm
- funcţii care realizează prelucrări asupra unei familii de clase, fără a modifica efectiv structura claselor

Structura:

- Element (sau *Observable*) şi Visitor
- Element: are o metodă *accept* care poate primi ca argument un *Visitor*
  - Metoda *accept* apelează metoda *visit* din *Visitor; Element-ul* se transmite pe sine însuşi ca argument al metodei *visit*

Exemplu

- Algoritm pentru parcurgerea elementelor unui graf
- Mai multe operaţii ce pot fi realizate în timpul acestei parcurgeri prin furnizarea a diferite tipuri de vizitatori care să interacţioneze cu elementele grafului
- Are la bază tipul dinamic atât al elementelor grafului cât şi al vizitatorilor – double dispatch!
- Double dispatch - Metoda apelată este determinată la *runtime* de doi factori!

Double Dispatch

Atunci când metoda *accept* e apelată în program, implementarea ei este determinată pe baza:

- Tipului dinamic (la execuţie) al *Element-ului* şi
- Tipului static al *Visitor-ului*.

Atunci când metoda asociată *visit* e apelată în program, implementarea ei este determinată pe baza:

- Tipului dinamic al *Visitor-ului* şi
- Tipului static al *Element-ului* aşa cum este el cunoscut din implementarea metodei *accept*, care înseamnă de fapt tipul dinamic al *Element-ului*

Prin urmare, implementarea metodei *visit* este determinată pe baza:

- Tipului dinamic al *Element-ului şi*
- Tipului dinamic al *Visitor-ului*

Exemplul 1

- Afişarea conţinutului unui arbore de noduri, care în acest caz descrie componentele unei maşini.
- În loc să scriem metoda "print" pentru fiecare subclasă (Wheel, Engine, Body şi Car), avem o singură clasă (CarElementPrintVisitor) ce realizează acţiunea de afişare dorită.
- Deoarece diferitele subclase necesită acţiuni uşor diferite pentru a afişa corect, CarElementDoVisitor determină acţiunile pe baza clasei argumentului primit de către el.

Implementare

```
interface ICarElementVisitor {
    void visit(Wheel wheel);
    void visit(Engine engine);
    void visit(Body body);
    void visit(Car car);
}

interface ICarElement {
    void accept(ICarElementVisitor visitor);
}

class Wheel implements ICarElement {
    private String name;
    public Wheel(String name) {
        this.name = name;
    }
    public String getName() {
        return this.name;
    }
```

Implementare

```
    public void accept(ICarElementVisitor visitor) {
        /*
         * accept(ICarElementVisitor) din Wheel implementeaza
         * accept(ICarElementVisitor) din ICarElement, astfel incat
         * apelul lui accept se face prin dynamic binding (la execuţie).
         * Acesta poate fi considerat “the first dispatch”.
         * Decizia de a apela  visit(Wheel) (si nu visit(Engine) etc.)
         * poate fi luată la compilare deoarece 'this' este cunoscut în
         * momentul compilării ca fiind un Wheel. Pe lângă aceasta, fiecare
         * implementare  a lui ICarElementVisitor implementează
         * visit(Wheel), care este o altă decizie ce este luată în momentul
          * execuţiei. Acesta poate fi considerat “the second dispatch”.
         */
        visitor.visit ( this );
    }
}
class Engine implements ICarElement {
    public void accept(ICarElementVisitor visitor) {
        visitor.visit ( this );
    }
}
```

Implementare

```
class Body implements ICarElement {
    public void accept(ICarElementVisitor visitor) {
        visitor.visit ( this );
    }
}
 class Car implements ICarElement {
    ICarElement[ ] elements;
    public Car() {
         elements = new ICarElement[] { new Wheel("front left"),
            new Wheel("front right"), new Wheel("back left") ,
            new Wheel("back right"), new Body(), new Engine() };
    }
    public void accept(ICarElementVisitor visitor) {
        for( ICarElement elem : elements) {
             elem.accept (visitor);
        }
        visitor.visit ( this );
    }
}
```

Implementare

```
class CarElementPrintVisitor implements ICarElementVisitor {

     public void visit (Wheel wheel) {
        System.out.println("Visiting " + wheel.getName() + " wheel");
    }

    public void visit(Engine engine) {
        System.out.println("Visiting engine");
    }

    public void visit(Body body) {
        System.out.println("Visiting body");
    }

    public void visit(Car car) {
        System.out.println("Visiting car");
    }
}
```

Implementare

```
class CarElementDoVisitor implements ICarElementVisitor {
    public void visit(Wheel wheel) {
        System.out.println("Kicking my " + wheel.getName() + " wheel");
    }
    public void visit(Engine engine) {
        System.out.println("Starting my engine");
    }
    public void visit(Body body) {
        System.out.println("Moving my body");
    }
    public void visit(Car car) {
        System.out.println("Starting my car");
    }
}
public class VisitorDemo {
    public static void main(String[] args) {
        ICarElement car = new Car();
        car.accept(new CarElementPrintVisitor());
        car.accept(new CarElementDoVisitor());
    }
}
```

Output

```
Visiting front left wheel
Visiting front right wheel
Visiting back left wheel
Visiting back right wheel
Visiting body
Visiting engine
Visiting car
Kicking my front left wheel
Kicking my front right wheel
Kicking my back left wheel
Kicking my back right wheel
Moving my body
Starting my engine
Starting my car
```

Observaţie

- O abordare mai flexibilă a acestui pattern este de a crea o *clasă wrapper* care implementează interfaţa care defineşte metoda *accept*.
- Această clasă conţine o referinţă la ICarElement care poate fi iniţializată prin constructor.
- Această abordare evită necesitatea ca fiecare element să implementeze o interfaţă.

```
class Wrapper implements ICarElement {
     ICarElement el;
    public Wrapper(ICarElement el){
          this.el=el;}
    public void accept(ICarElementVisitor visitor) { }
}
```

Exemplul 2 - Implementare

```
interface Visitor {
        public void visit(Student e);
        public void visit(Profesor b);
}
interface Visitable {
        public void accept(Visitor v);
}
class Student implements Visitable {
        ...
        public void accept(Visitor v) {
                v.visit(this);
        }
}
class Profesor implements Visitable {
        ...
        public void accept(Visitor v) {
                v.visit(this);
        }
}
```

Implementare

```
public class Test {
        public static void main(String[] args) {
                ...
                Visitor v = new ConcreteVisitor();
                for (Visitable e : persoane) // colectie de Persoane
                        e.accept ( v );
        }
}
```

- vizitator ce afişează datele fiecărei persoane (Student sau Profesor):

```
public class ConcreteVisitor implements Visitor {
        public void visit(Student s) {
                System.out.println(s.nume() + " " + s.grupa());
        }
        public void visit(Profesor p) {
                System.out.println(p.nume() + " " +  + p.materia());
        }
}
```